Predicting cp.async Pipeline Effectiveness from Memory-Subsystem: A Cross-GPU Study

Students: Yuyang Feng, Chenxin Geng, Xiangchen Song, Tian Xia
University of Michigan - EECS 570 - Parallel Computer Architecture - WN2026
Professor Satish Narayanasamy

MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

Introduction

NVIDIA cp.async (Ampere+) enables asynchronous global→shared memory copies that overlap with computation. CUTLASS, FlashAttention-3, and Triton all rely on multi-stage cp.async pipelines with depth *S* as a key tuning knob.

- **The Problem**: Practitioners tune *S* on one GPU and assume it transfers. But pipeline effectiveness depends on L2 capacity, memory latency, and shared memory per SM — which vary dramatically across GPU generations. No existing autotuner uses architecture parameters to prune the pipeline-depth search space.
- **Research Question**: Can a parameter-free model predict cp.async speedup from architecture constants alone — and reduce autotuning cost in practice?

Hypotheses

- **H1 (L2-Capacity Driven) ✅ SUPPORTED**
  Speedup inflects when concurrent working set $W_{conc} > C_{L2}^{eff}$
  L40S (112 MB): minimal benefit. A40 (8 MB): largest gain.
- **H2 (Ridge-Point Driven): ⚠️ PARTIAL SUPPORTED**
  Low ridge-point GPUs benefit most from pipelining.
  Ridge point alone insufficient: must combine with L2 size and latency
- **H3 (Latency–Occupancy Tension): ✅ SUPPORTED**
  Optimal *S* balances latency hiding vs. occupancy loss.
  A40 shows non-monotonic S(N).

Method: Parameter Free Model

Inputs (from pointer-chase microbenchmark)

C_{L2}^{eff}	Wffective L2 capacity (bytes)	Which GPU/problem size will benefit
L_{L2}	L2 cache latency (cycles)	How much benefit to expect
L_{DRAM}	DRAM latency (cycles)	Which pipeline depth *S* is optimal

Step 1: Effective memory latency

Estimate how expensive each memory load is by blending L2 and DRAM latencies based on how much of the working set fits in the GPU's L2 cache

$$h = min(1, \frac{C_{L2}^{eff}}{W_{conc}}) \qquad L_{eff} = h \cdot L_{L2} + (1 - h) \cdot L_{DRAM}$$

Step 2: Compute time with empirical floor

Estimate the compute time per pipeline stage from FLOP count and GPU throughput, with a measured minimum floor to account for synchronization and scheduling overhead.

$$T_{comp} = max(T_{FLOPS}, 280\ cycles)$$

*The 280-cycle floor captures _syncthreads + cp.async commit/wait overhead

Step 3: Predicted speedup

Combine latency and compute time to determine how much of the memory stall can be hidden by pipelining, penalized by any occupancy loss from staging extra shared memory buffers

$$\widehat{S_{speed}} = \frac{max(T_{comp}, L_{eff})}{max(T_{comp}, \frac{L_{eff}}{S})} \times \left(\frac{Occ(S)}{Occ(1)}\right)^{0.15} \qquad \alpha = 0.15$$

Experimental Setup & Validation

Evaluated Kernels & Configurations

Kernel	Version	Description
GEMM	V1, V3	C = AB, Compute-bound N ∈ {256, 512, 1024, 2048, 3072, 4096, 6144, 8192} tile ∈ {6, 32, 64, 128}
Stencil	V1, V3	1-D, Memory-bound (AI ≈ 5) L ∈ {2^{16}, 2^{18}, 2^{20}, 2^{22}, 2^{24}, 2^{26}}

*V1: standard shared-memory kernel (synchronous, baseline)
*V3: same kernel but with cp.async pipelining at depth S=2, 3, or

Hardware: 5 GPU Architecture

	V100	A40	A100 MIG	L40S	H100
Arch	Volta	Ampere	Ampere	Ada	Hopper
SMs	80	84	42	142	132
BW (GB/s)	900	696	968	864	3350
SMEM/SM	96 K	100 K	164 K	100 K	228 K
Ridge	17.4	53.7	7.9	52.9	20.0
L2 nominal	6 MB	6 MB	20 MB	96 MB	50 MB
L2 effective	**6.5 MB**	**6 MB**	**21 MB**	**96 MB**	**26 MB**

Key: L2 effective ≈ nominal on most GPUs (within ~5%). Only H100 differs: 50 → 26 MB (slice locality).

Model Accuracy (MAPE)

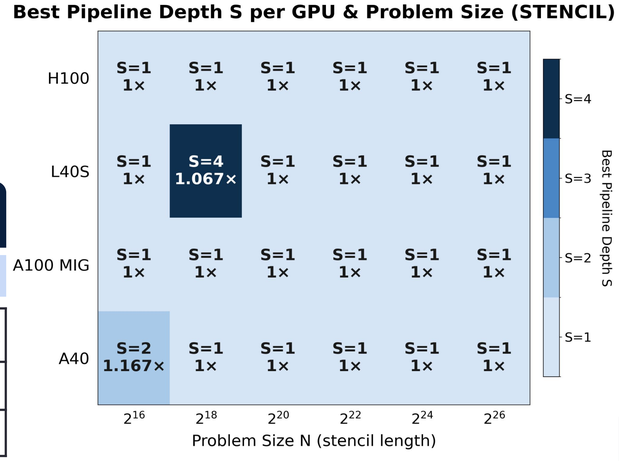
GPU	GEMM MAPE	Stencil MAPE
A40	10.4%	4.1%
A100 MIG	9.2%	6.7%
L40S	6.5%	6.3%
H100	9.7%	3.7%
Overall	**9.0%**	**5.2%**

Leave-one-out: held-out GPU < 15% MAPE. Zero fitted parameters.

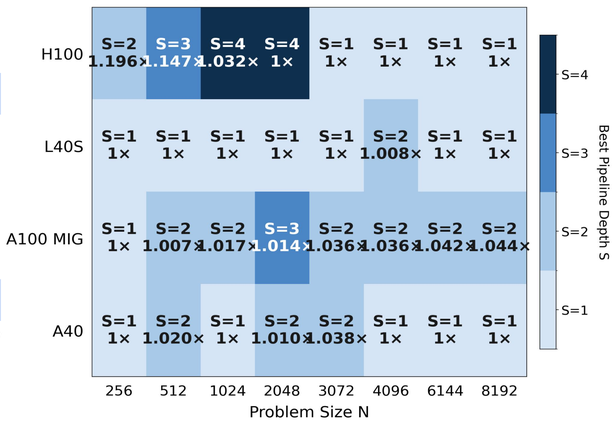
Results

Best Pipeline Depth

Best Pipeline Depth S per GPU & Problem Size (STENCIL)

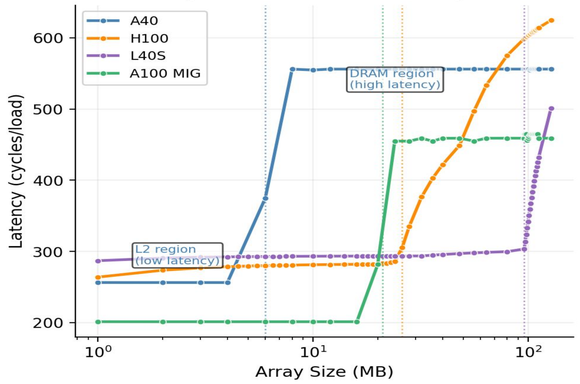


Best Pipeline Depth S per GPU & Problem Size (GEMM)

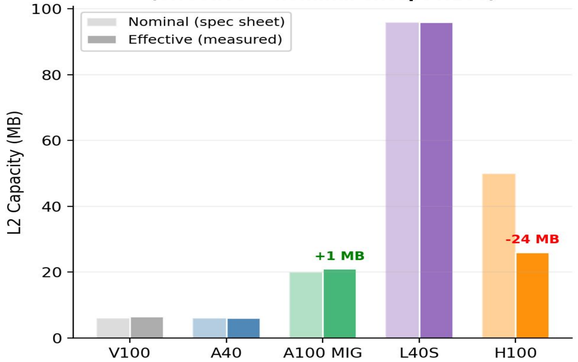


L2 Characterization (Pointer-Chase Microbenchmark)

Pointer-Chase Latency Curves (dotted = effective L2 cliff)

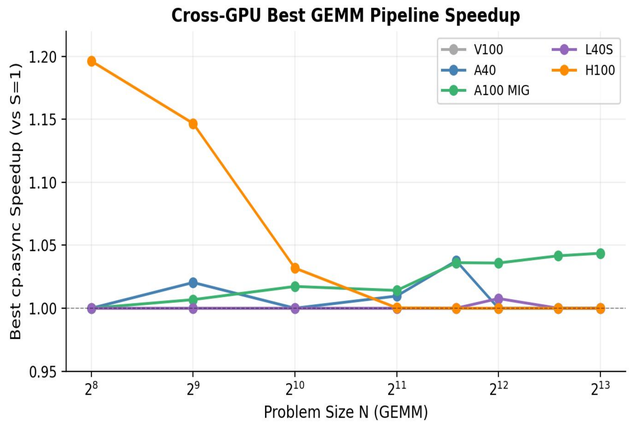


Nominal vs Effective L2 Capacity (effective ≈ nominal except H100)



Discussion

Cross-GPU Best GEMM Pipeline Speedup



1. Effective L2 ≠ Nominal

H100: only 26 MB (not 52 MB) due to L2 slice locality — first published characterization. The other 4 GPUs (V100, A40, A100 MIG, L40S) all show effective ≈ nominal.

2. 280-cycle per-stage floor

Even 28 FLOP-cycle stencils incur ≥280 cycles of sync overhead.

Reduced H100 stencil MAPE: 146% → 3.7%.

3. Sub-linear occupancy loss (α = 0.15)

3→1 blocks/SM costs ~16%, not 67%.

4. Parameter-free model achieves ~9% MAPE

across 4 GPU architectures, 2 kernels, zero fitted coefficients.

5. Pipeline tuning is not portable

Peak GEMM speedup ranges from 1.0× (L40S) to 1.20× (H100) on identical code.